TP Inférence causale en épidémiologie

David Hajage

Sommaire

Introduction **4**
Scénario . . . 4
Deux questions causales . . . 4
Structure du TP . . . 4
Comment utiliser ce TP . . . 5

Présentation des données **6**
Présentation . . . 6
Chargement et exploration . . . 7
Analyse brute (non ajustée) . . . 7
Ce que nous observons, ce que nous voulons estimer . . . 8

1 G-computation **10**
1.1 Objectif - Question 1 . . . 10
1.2 Note sur la session R . . . 10
1.3 Étape 0 - Créer la base *baseline* . . . 11
1.4 Étape 1 - Estimer les modèles de résultat . . . 11
1.5 Étape 2 - Prédire les devenirs contrefactuels . . . 12
1.6 Étape 3 - Calculer l'effet causal moyen (ATE) . . . 13
1.7 Interprétation . . . 13
1.8 BONUS - Intervalle de confiance par bootstrap . . . 14
1.9 BONUS 2 - G-computation sur critère censuré . . . 15
Pourquoi changer d'approche ? . . . 15
Principe . . . 15
Étape 1 - Un modèle de Cox avec interaction . . . 16
Étape 2 - Hazard cumulatif de base et prédicteurs linéaires individuels . . . 16
Étape 3 - Courbes de survie individuelles et marginalisation . . . 17
Étape 4 - Visualisation des courbes de survie contrefactuelles . . . 18
Étape 5 - Différence de survie à $t = 3$ ans . . . 19
Intervalle de confiance par bootstrap (modèle de Cox) . . . 20

2 IPTW **22**
2.1 Objectif - Question 1 (suite) . . . 22
2.2 Étape 1 - Estimer le score de propension . . . 22
2.3 Étape 2 - Calculer les poids IPTW . . . 23
2.4 Étape 3 - Vérifier l'équilibre . . . 24
2.5 Étape 4 - Analyse de survie pondérée (Kaplan-Meier) . . . 26
2.6 Comparaison G-computation vs IPTW . . . 26
2.7 BONUS - Intervalle de confiance par bootstrap . . . 28

3 IPCW **31**
3.1 Objectif - Question 2 . . . 31
3.2 Note sur la session R . . . 31
3.3 Étape 1 - Définir les deux groupes de stratégie . . . 31
3.4 Étape 2 - Censure artificielle dans le groupe A0 = 1 . . . 32
3.5 Étape 3 - Modèle de déviation et poids IPCW (groupe A0 = 1) . . . 33
3.6 Étape 4 - Répéter pour le groupe A0 = 0 . . . 34
3.7 Étape 5 - Poids combinés IPTW × IPCW . . . 35
3.8 Étape 6 - Analyse de survie per-protocol . . . 36
3.9 Interprétation . . . 36
3.10 BONUS - Intervalle de confiance par bootstrap . . . 37
3.11 BONUS 2 - Clonage, censure, pondération : IPCW seul . . . 40
Comparaison des poids individu × visite . . . 41
Comparaison des courbes de survie . . . 42

4 Conclusion **44**
4.1 Comparaison graphique . . . 44
4.2 Récapitulatif des estimations . . . 44
4.3 Discussion . . . 45
Ce que nous apprenons de chaque analyse . . . 45

Introduction

Bienvenue dans ce TP sur l'**inférence causale en épidémiologie** !

Scénario

Nous disposons d'une **étude de cohorte observationnelle** dont l'objectif est d'évaluer l'effet d'une exposition A (par exemple un traitement médical) sur la **survie à 3 ans**.

Les patients sont suivis pendant 3 ans avec des visites annuelles. À chaque visite, leur statut d'exposition peut changer : un patient initialement exposé peut arrêter, et inversement. Un facteur de confusion L, dépendant du temps, est mesuré à chaque visite. Une covariable initiale X (non dépendante du temps) est également disponible.

Deux questions causales

Ce TP vous guidera pour répondre à **deux questions causales distinctes** :

i Question 1 - Effet de l'exposition initiale (analogue-ITT)

Quelle serait la survie si tout le monde avait été exposé dès le début de l'étude ($A_0 = 1$), et quelle serait la survie si personne ne l'avait été ($A_0 = 0$) ?
Cette question porte sur l'*initiation* de l'exposition au temps 0, indépendamment de ce qui se passe ensuite (les patients peuvent arrêter ou débuter le traitement par la suite).
→ Abordée en **Partie 1** (G-computation) et **Partie 2** (IPTW)

i Question 2 - Effet de l'exposition maintenue (analogue per-protocol)

Quelle serait la survie si tout le monde avait été exposé et l'était resté tout au long du suivi ($\bar{a} = 1$), et si personne n'avait jamais été exposé ($\bar{a} = 0$) ?
Cette question porte sur le *maintien* de l'exposition tout au long du suivi. Elle peut être traitée en censurant artificiellement les individus qui dévient de leur stratégie d'exposition.
→ Abordée en **Partie 3** (IPTW + IPCW)

Structure du TP

Partie	Méthode	Question	Durée estimée
Présentation	Analyse brute (Kaplan-Meier)	Description	~10 min
Partie 1	**G-computation**	Q1 - effet de l'exposition initiale	~20 min
Partie 2	**IPTW**	Q1 - même objectif, approche différente	~25 min
Partie 3	**IPTW + IPCW**	Q2 - effet de l'exposition maintenue	~30 min
Conclusion	Comparaison des résultats	Synthèse	~5 min

Les durées annoncées sont approximatives, vous pourrez terminer chez vous ce que vous n'auriez pas eu le temps de faire aujourd'hui. Vous pouvez télécharger le pdf de ce TP et le fichier de données `df.csv` *via les liens tout en bas de cette page.*

Comment utiliser ce TP

Les exercices utilisent **WebR** : le code R s'exécute directement dans votre navigateur, sans installation. Si vous préférez, vous pouvez télécharger le fichier `df.csv` et exécuter les commandes dans une session locale de RStudio.

- Chaque bloc de code peut être modifié et exécuté avec le bouton **Run Code** (raccourci : `Ctrl+R` sur Windows, `Cmd+R` sur Mac)
- Les **solutions** sont là pour être consultées librement et sans hésitation. L'objectif de ce TP n'est pas de retrouver le code depuis une page blanche, mais de **suivre et comprendre chaque étape** : lisez-les comme un exemple commenté, exécutez-les, modifiez-les. Certaines analyses mobilisent des compétences R avancées, et il est tout à fait normal de s'appuyer sur les solutions.
- Les variables créées sur une page ne sont **pas disponibles** sur les pages suivantes ; elles sont reconstruites automatiquement dans chaque section (voir *Note sur la session R* au début de chaque partie).

Présentation des données

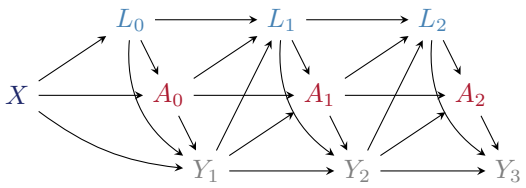
Présentation

Dans ce TP, nous utiliserons un jeu de données simulées (`df.csv`) contenant des informations sur **2000 individus** suivis dans le cadre d'une étude de cohorte observationnelle. L'objectif est d'évaluer l'effet d'un traitement A (exposé/non exposé, pouvant changer au cours du temps) sur la **mortalité à 3 ans**.

Les données sont au **format long** (*counting process*) : chaque individu contribue une ligne par période d'observation (ici, une ligne par visite annuelle).

Variable	Description
`id`	Identifiant patient
`T.start`	Début de la période d'observation (en années)
`T.stop`	Fin de la période d'observation (en années)
`A`	Traitement (0/1), **dépendant du temps**
`D`	Décès survenu avant `T.stop` (0/1)
`L`	Facteur de confusion **dépendant du temps** (continu)
`X`	Facteur de confusion **initial** (binaire 0/1, non dépendant du temps)

On suppose que les relations entre les variables suivent le DAG (*directed acyclic graph*) en temps discret ci-dessous, où Y_t désigne l'indicateur de décès à la période t ($t = 1, 2, 3$) :



Lecture du DAG :

- X (bleu foncé) - facteur de confusion **initial** : influence A_0, L_0 **et** Y_1.
- L_t (bleu clair) - facteur de confusion **dépendant du temps** : influencé par le traitement passé A_{t-1} et l'état de santé Y_t, il prédit à son tour A_t et Y_{t+1}.
- A_t (rouge) - exposition à chaque période, influencée par X, L_t et les valeurs passées.
- Y_t (gris) - indicateur de décès, influencé par A_{t-1} et L_{t-1}.

Chargement et exploration

Chargez les données et affichez les premières lignes. Vérifiez les dimensions de la base et le nombre d'individus distincts.

```
#| context: interactive
library(survival)
library(dplyr)
df <- read.csv("df.csv")

## Affichez les premières lignes, les dimensions, le nombre d'individus et de décès
## Affichez ensuite toutes les lignes des 3 premiers individus pour voir le format long
```

{**Correction :**}

```
library(survival)
library(dplyr)
df <- read.csv("df.csv")

head(df)
dim(df)                    # dimensions de la base
length(unique(df$id))      # nombre d'individus
sum(df$D)                  # nombre total de décès
df[df$id %in% 1:3, ]       # 3 premiers individus (plusieurs lignes chacun)
```

La base comporte 4376 lignes, 8 colonnes, 2000 individus distincts et 409 décès observés.

> **Note**
>
> **Format long : chaque individu a autant de lignes qu'il a eu de périodes de suivi (en général 1 à 3, selon s'il est décédé ou perdu de vue précocement). La colonne `T.start`/`T.stop` délimite chaque période. Le dernier enregistrement par individu indique son statut final (D) et son temps total de suivi.**

Analyse brute (non ajustée)

Avant tout ajustement, comparons les courbes de survie selon le traitement initial A_0 (valeur de A à la première visite).

Créez la variable `A0` (valeur initiale de A pour chaque individu), puis tracez les courbes de Kaplan-Meier selon `A0`.

```
#| context: interactive
## Créez A0 (première valeur de A pour chaque individu), tracez le Kaplan-Meier selon A0
## Extrayez ensuite la survie estimée à 3 ans dans chaque groupe
df <- df |>
  group_by(id) |>
  mutate(A0 = first(A)) |>
  ungroup()
```

{Correction :}

```
## A0 : valeur de A à la première visite, propagée à toutes les lignes de l'individu
df <- df |>
  group_by(id) |>
  mutate(A0 = first(A)) |>
  ungroup()

## Surv(T.start, T.stop, D) : format comptage pour données en format long
## (plusieurs lignes par individu, une par période de suivi)
km_brut <- survfit(Surv(T.start, T.stop, D) ~ A0, data = df)

plot(km_brut,
     col  = c("#1D2769", "#AC182E"), lwd = 2,
     xlab = "Temps (années)", ylab = "Probabilité de survie",
     main = "Kaplan-Meier brut selon A0 (non ajusté)")
legend("bottomleft",
       legend = c("A0 = 0 (non-exposés)", "A0 = 1 (exposés)"),
       col = c("#1D2769", "#AC182E"), lwd = 2)

## Survie à 3 ans
summary(km_brut, times = 3)
```

Que constatez-vous ? Cette différence brute a-t-elle une interprétation causale ?

Réponse

On observe une différence de survie entre les groupes $A_0 = 0$ et $A_0 = 1$. Cependant, cette différence n'a pas d'interprétation causale car les deux groupes ne sont pas comparables : les individus traités ($A_0 = 1$) et non traités ($A_0 = 0$) diffèrent sur X et L_0, qui sont à la fois des prédicteurs de l'exposition et du décès (facteurs de confusion).
Les parties suivantes montrent comment corriger ce biais par G-computation et IPTW.

Ce que nous observons, ce que nous voulons estimer

L'analyse brute met en évidence une différence de survie entre les groupes $A_0 = 0$ et $A_0 = 1$. Mais cette différence n'a pas d'interprétation causale directe : les deux groupes ne sont pas comparables à l'inclusion (les individus exposés et non exposés diffèrent sur X et L_0, qui prédisent à la fois l'exposition et le décès).

Les parties suivantes vont estimer deux effets causaux distincts à partir de ces données :

— **Parties 1 et 2 : effet de l'*initiation* de l'exposition au temps 0, ajusté sur les facteurs de confusion initiaux (d'abord par G-computation, puis par IPTW).**
— **Partie 3 : effet du *maintien* de l'exposition tout au long du suivi, en traitant les déviations de stratégie comme une censure artificielle (IPTW + IPCW).**

1 G-computation

1.1 Objectif - Question 1

Cette partie répond à la Question 1 : quel serait l'effet d'une exposition initiale universelle ($a_0 = 1$ pour tous) par rapport à une absence totale d'exposition ($a_0 = 0$ pour tous) ?

On souhaite estimer les courbes de survie contrefactuelles :

$$S^{a_0=1}(t) = P(T^{a_0=1} > t) \quad \textbf{et} \quad S^{a_0=0}(t) = P(T^{a_0=0} > t)$$

La méthode utilisée ici est la G-computation (standardisation par régression) : on modélise le résultat Y en fonction de l'exposition et des covariables, puis on prédit les devenirs contrefactuels pour l'ensemble de la population.

> **Note**
>
> **Simplification pédagogique : pour cette partie, le critère de jugement est traité comme binaire (décès en fin de suivi, oui/non) : on estime $\widehat{P}(D = 1 \mid a_0 = 1)$ et $\widehat{P}(D = 1 \mid a_0 = 0)$, c'est-à-dire deux proportions de décès contrefactuelles. On peut effectuer une G-computation pour estimer directement la courbe de survie complète, mais c'est techniquement plus complexe (une section bonus en bas de cette page en montre la mise en œuvre, à consulter et à faire chez vous). La Partie 2 traitera le critère censuré directement via l'IPTW.**

1.2 Note sur la session R

Les variables suivantes sont disponibles dans vos chunks interactifs (construites dans le contexte de setup) :

- **`df` : base de données au format long, avec `A0` (exposition initiale, valeur de `A` à $t = 0$) et `L0` (facteur de confusion initial, valeur de `L` à $t = 0$)**

Les premières lignes de `df` illustrent ce format long et montrent les colonnes `A0` et `L0` :

```
# A tibble: 6 x 9
     id T.start T.stop     A     D      L     X    A0    L0
  <int>   <int>  <dbl> <int> <int>  <dbl> <int> <int> <dbl>
1     1       0   1         0     0 -1.43      0     0 -1.43
2     1       1   2         1     0 -0.603     0     0 -1.43
3     1       2   2.21      1     0  1.10      0     0 -1.43
4     2       0   1         1     0  2.10      0     1  2.10
5     2       1   2         0     0 -1.39      0     1  2.10
6     2       2   2.5       0     0 -1.96      0     1  2.10
```

1.3 Étape 0 - Créer la base *baseline*

La G-computation s'applique sur une base une ligne par individu, avec les valeurs initiales des covariables et le statut final.

Créez `df_base` contenant : `id`, `A0`, `L0`, `X`, `time` (temps total de suivi = dernière valeur de `T.stop`), et `D` (statut final = dernière valeur de `D`).

```
#| context: interactive
## Créez df_base : une ligne par individu, avec A0, L0, X (valeurs initiales),
## time (dernière valeur de T.stop) et D (statut final, dernière valeur de D)
```

{Correction :}

```
## On résume df (format long) en une ligne par individu :
## - first() pour les covariables mesurées à t=0
## - last() pour le temps de suivi total et le statut final
df_base <- df |>
  group_by(id) |>
  summarise(
    A0   = first(A),     # exposition initiale
    L0   = first(L),     # facteur de confusion initial
    X    = first(X),     # covariable initiale
    time = last(T.stop), # temps total de suivi
    D    = last(D)       # décès (0/1) : statut en fin de suivi
  )

head(df_base)
nrow(df_base)  # doit valoir le nombre d'individus distincts
```

La base comporte 2000 lignes (une par individu). La variable `D` indique si l'individu est décédé avant la fin du suivi (3 ans).

1.4 Étape 1 - Estimer les modèles de résultat

La G-computation ajuste le modèle d'outcome (D) séparément selon le groupe d'exposition A_0, en contrôlant pour les facteurs de confusion X et L_0.

Estimez deux modèles de régression logistique sur `D` : `mod1` sur les individus avec `A0 = 1`, `mod0` sur les individus avec `A0 = 0`.

```
#| context: interactive
## Modèle chez les exposés (A0 = 1)

## Modèle chez les non-exposés (A0 = 0)
```

{Correction :}

```r
## Régression logistique sur D dans chaque groupe d'exposition séparément :
## on ajuste sur X et L0 pour contrôler la confusion
## (deux modèles séparés = équivalent à une interaction complète avec A0)
mod1 <- glm(D ~ X + L0,
            data   = df_base[df_base$A0 == 1, ],
            family = binomial)  # critère binaire -> famille binomiale (logit)

mod0 <- glm(D ~ X + L0,
            data   = df_base[df_base$A0 == 0, ],
            family = binomial)

summary(mod1)
summary(mod0)
```

1.5 Étape 2 - Prédire les devenirs contrefactuels

Pour chaque individu (quelle que soit son exposition réelle), on prédit le risque de décès comme s'il avait été exposé (via `mod1`) et comme s'il n'avait pas été exposé (via `mod0`).

Calculez les vecteurs `y1` et `y0` : probabilités de décès prédites sous $a_0 = 1$ ***et*** $a_0 = 0$***.***

```r
#| context: interactive
## Utiliser predict() pour prédire pour TOUS les individus le risque de décès
## sous chaque scénario : y1 = P(D=1 | A0=1) via mod1, y0 = P(D=1 | A0=0) via mod0
## Passer newdata = df_base sans filtre : on prédit pour tout le monde,
## quel que soit son A0 observé
```

{Correction :}

```r
## Probabilité de décès si tout le monde avait A0 = 1
y1 <- predict(mod1, newdata = df_base, type = "response")

## Probabilité de décès si tout le monde avait A0 = 0
y0 <- predict(mod0, newdata = df_base, type = "response")

head(data.frame(id = df_base$id, A0_obs = df_base$A0,
                pred_A0_1 = round(y1, 3),
                pred_A0_0 = round(y0, 3)))
```

1.6 Étape 3 - Calculer l'effet causal moyen (ATE)

Calculez $\widehat{E}(Y^1) = \overline{y_1}$, $\widehat{E}(Y^0) = \overline{y_0}$, puis l'ATE et le RR.

```
#| context: interactive
## Moyennez y1 et y0 sur toute la population pour obtenir E(D^1) et E(D^0)
## Calculez ensuite l'ATE (différence de risque) et le RR (rapport de risque)
```

{Correction :}

```
E_Y1 <- mean(y1)
E_Y0 <- mean(y0)

cat("E(D^1)                    =", round(E_Y1, 3), "\n")
cat("E(D^0)                    =", round(E_Y0, 3), "\n")
cat("ATE = E(D^1) - E(D^0)     =", round(E_Y1 - E_Y0, 3), "\n")
cat("RR  = E(D^1) / E(D^0)     =", round(E_Y1 / E_Y0, 3), "\n")
```

On obtient : $\widehat{E}(Y^1) \approx$ 0.125, $\widehat{E}(Y^0) \approx$ 0.26, $\widehat{ATE} \approx$ -0.135.

1.7 Interprétation

Que signifie cet ATE ? Comment le comparez-vous à l'analyse brute de la page précédente ?

Réponse

L'ATE estimé par G-computation est la différence de risque de décès entre les scénarios contrefactuels "tout le monde exposé" et "personne exposé", après ajustement sur X et L_0.
Contrairement à l'analyse brute, cette estimation tient compte du fait que les individus exposés et non exposés ne sont pas comparables à l'inclusion.
Points à retenir :

- **On traite le décès comme binaire, en ignorant l'information temporelle (voir le bonus en bas de page pour la version survie complète).**
- **On n'ajuste que sur les confondeurs initiaux, ce qui est suffisant ici : la question porte sur l'exposition initiale A_0. La Partie 3 traitera les confondeurs dépendants du temps dans le cadre de l'analyse per-protocol.**
- **L'estimation repose entièrement sur la bonne spécification du modèle d'outcome D.**

La Partie 2 répondra à la même Question 1 en utilisant une approche différente (IPTW), ce qui permettra un contrôle croisé des deux estimations.

1.8 BONUS - Intervalle de confiance par bootstrap

> **Note**
>
> **Cette section est à faire chez vous, à votre rythme. Elle n'est pas traitée en TP.**

Le bootstrap sert ici uniquement à estimer un intervalle de confiance autour de l'ATE calculé à l'Étape 3 : il ne modifie pas l'estimation elle-même.

Estimez un intervalle de confiance à 95% par bootstrap *($B = 200$* ***ré-échantillons).***

```
#| context: interactive
set.seed(123)
B <- 200
ate_boot <- numeric(B)

for (b in 1:B) {
  ## 1. Ré-échantillonner df_base avec remise

  ## 2. Ajuster mod1 et mod0

  ## 3. Prédire y1, y0

  ## 4. Calculer l'ATE

}
quantile(ate_boot, c(0.025, 0.975))
```

{Correction :}

```
set.seed(123)
B <- 200
ate_boot <- numeric(B)

for (b in 1:B) {
  ## Ré-échantillonnage avec remise (même taille que df_base)
  idx  <- sample(nrow(df_base), replace = TRUE)
  df_b <- df_base[idx, ]

  ## Réajuster les deux modèles sur l'échantillon bootstrap
  m1 <- glm(D ~ X + L0, data = df_b[df_b$A0 == 1, ], family = binomial)
  m0 <- glm(D ~ X + L0, data = df_b[df_b$A0 == 0, ], family = binomial)

  ## Prédire pour tous les individus du bootstrap sous chaque scénario
  y1_b <- predict(m1, newdata = df_b, type = "response")
  y0_b <- predict(m0, newdata = df_b, type = "response")

  ## Stocker l'ATE de cet échantillon bootstrap
  ate_boot[b] <- mean(y1_b) - mean(y0_b)
}

## L'ATE est celui estimé à l'Étape 3 (E_Y1 - E_Y0), pas la moyenne des bootstrap
```

```r
cat("ATE    :", round(E_Y1 - E_Y0, 3), "\n")
cat("IC 95% :", round(quantile(ate_boot, c(0.025, 0.975)), 3), "\n")
```

1.9 BONUS 2 - G-computation sur critère censuré

> **Note**
>
> **Cette section est à faire chez vous, à votre rythme. Elle n'est pas traitée en TP. Elle montre comment étendre la G-computation vue précédemment pour traiter correctement le critère de jugement censuré.**

Pourquoi changer d'approche ?

La G-computation de la partie principale modélise D comme un indicateur binaire (décédé avant $t = 3$ ans, oui/non). Deux limites importantes :

1. **Information temporelle perdue : on sait *si* l'individu est décédé, mais on ignore *quand*.**
2. **Censure traitée comme un non-événement : un individu perdu de vue à $t = 1$ an est traité de la même façon qu'un individu suivi 3 ans sans événement.**

On peut effectuer une G-computation pour estimer directement des courbes de survie contrefactuelles :

$$\hat{S}^{a_0=1}(t) = P(T^{a_0=1} > t) \qquad \textbf{et} \qquad \hat{S}^{a_0=0}(t) = P(T^{a_0=0} > t)$$

Principe

Deux approches sont possibles : estimer deux modèles de Cox séparément (un chez les exposés, un chez les non-exposés), ou estimer un seul modèle incluant une interaction entre A_0 et les covariables (X, L_0). Nous retenons la seconde, plus synthétique.

Le modèle estimé sur l'ensemble de la population est :

$$h(t; A_0, X, L_0) = h_0(t) \cdot \exp(\beta_1 A_0 + \beta_2 X + \beta_3 L_0 + \beta_{12} A_0 X + \beta_{13} A_0 L_0)$$

Pour obtenir les courbes contrefactuelles, on prédit le prédicteur linéaire $\hat{\text{lp}}_i$ pour chaque individu dans deux bases contrefactuelles (tous exposés : $A_0 = 1$, tous non-exposés : $A_0 = 0$), puis on calcule la survie individuelle :

$$\hat{S}_i^{a_0}(t) = \exp\left(-\hat{H}_0(t) \times e^{\hat{\text{lp}}_i^{a_0}}\right)$$

— $\hat{H}_0(t)$ **: hazard cumulatif de base (estimateur de Breslow), commun à toute la population**
— $e^{\hat{\text{lp}}_i^{a_0}}$ **: multiplicateur individuel de risque sous le scénario a_0**

La courbe de survie marginale (contrefactuelle) s'obtient en moyennant sur l'ensemble de la population :

$$\hat{S}^{a_0}(t) = \frac{1}{n}\sum_{i=1}^{n} \hat{S}_i^{a_0}(t)$$

Étape 1 - Un modèle de Cox avec interaction

> **Note**
>
> `df_base` doit avoir été créée lors de l'Étape 0 ci-dessus. Si nécessaire, relancez ce chunk d'abord.

Estimez un modèle de Cox sur l'ensemble de la population, avec une interaction entre A_0 et les covariables (X, L_0). Utilisez `coxph(Surv(time, D) ~ A0 * (X + L0), data = df_base)`.

```
#| context: interactive
## Modèle de Cox avec interaction A0 × (X, L0)
```

{Correction :}

```
## Un seul modèle de Cox avec interaction A0 × covariables.
## Hypothèse : les deux groupes partagent le même risque de base h0(t).
## Deux modèles séparés lèveraient cette contrainte.
## Surv(time, D) : time = temps de suivi total, D = indicateur de décès
mod_cox <- coxph(Surv(time, D) ~ A0 * (X + L0), data = df_base)

summary(mod_cox)
```

Étape 2 - Hazard cumulatif de base et prédicteurs linéaires individuels

L'estimation des courbes contrefactuelles nécessite deux ingrédients :

- **Le hazard cumulatif de base $\hat{H}_0(t)$: c'est une fonction en escalier qui augmente à chaque temps d'événement. On l'obtient avec `basehaz(mod_cox, centered = FALSE)`, qui renvoie un data frame avec les colonnes `time` et `hazard`.**
- **Le prédicteur linéaire individuel $\widehat{\text{lp}}_i^{a_0}$: calculé dans deux bases contrefactuelles : l'une où tous les individus ont $A_0 = 1$, l'autre où tous ont $A_0 = 0$. On l'obtient avec `predict(mod_cox, newdata = ..., type = "lp")`.**

Créez les bases contrefactuelles `df1` (A0=1 pour tous) et `df0` (A0=0 pour tous), extrayez le hazard cumulatif de base, et calculez les prédicteurs linéaires sous chaque scénario.

```
#| context: interactive
## Bases contrefactuelles
df1 <- df_base; df1$A0 <- 1
df0 <- df_base; df0$A0 <- 0

## Hazard cumulatif de base

## Prédicteurs linéaires sous chaque scénario
```

{Correction :}

```r
## Bases contrefactuelles
df1 <- df_base; df1$A0 <- 1
df0 <- df_base; df0$A0 <- 0

## Hazard cumulatif de base unique (estimateur de Breslow)
## centered = FALSE : hazard de base au niveau de référence (X=0, L0=0, A0=0)
## suppressWarnings : Cox avec interaction génère un avertissement sans danger
bh <- suppressWarnings(basehaz(mod_cox, centered = FALSE))
head(bh)  # time = temps d'événement, hazard = H0(t) cumulé

## Prédicteurs linéaires individuels sous chaque scénario contrefactuel
lp1 <- predict(mod_cox, newdata = df1, type = "lp")
lp0 <- predict(mod_cox, newdata = df0, type = "lp")

head(data.frame(
  id         = df_base$id,
  A0_obs     = df_base$A0,
  lp_si_A1   = round(lp1, 3),
  lp_si_A0   = round(lp0, 3)
))
```

Étape 3 - Courbes de survie individuelles et marginalisation

On dispose maintenant de tout ce qu'il faut pour calculer $\hat{S}_i^{a_0}(t)$ pour chaque individu i et chaque temps t, puis d'en faire la moyenne.

Astuce

`stepfun(knots, values)` crée une fonction en escalier à partir des nœuds et valeurs associées. Ici, `stepfun(bh1$time, c(0, bh1$hazard))` crée la fonction $\hat{H}_0^{(a_0=1)}(t)$: elle vaut 0 avant le premier événement, puis prend la valeur de Breslow correspondante à chaque nœud. On peut ensuite l'évaluer sur n'importe quelle grille de temps.
`outer(u, v)` calcule le produit extérieur de deux vecteurs : `outer(u, v)[i, j] = u[i] * v[j]`. Cela permet de calculer $-e^{\hat{\mathbf{lp}}_i} \times \hat{H}_0(t_j)$ pour tous les couples (i, j) d'un coup, sans

boucle, et d'obtenir directement la matrice des $n \times T$ valeurs $\log \hat{S}_i(t_j)$.

Calculez les courbes de survie marginales $\hat{S}^{a_0=1}(t)$ et $\hat{S}^{a_0=0}(t)$ sur une grille de temps.

```
#| context: interactive
## 1. Grille de temps

## 2. Hazard cumulatif de base évalué sur cette grille (stepfun)

## 3. Matrice des survies individuelles n × T (outer + exp)

## 4. Courbes marginales (colMeans)
```

{Correction :}

```
## 1. Grille de temps : tous les temps d'événement de df_base
t_grid <- sort(unique(df_base$time))

## 2. H0(t) unique évalué sur t_grid via une fonction en escalier
## stepfun crée une fonction en escalier : c(0, bh$hazard) indique que
## le hazard cumulatif vaut 0 avant le premier temps d'événement
H_fun  <- stepfun(bh$time, c(0, bh$hazard))
H_grid <- H_fun(t_grid)   # vecteur longueur T

## 3. Survie individuelle : S_i(t_j) = exp(-H0(t_j) * exp(lp_i))
##    outer(-exp(lp), H_grid)[i, j] = -exp(lp[i]) * H_grid[j]
S1_mat <- exp(outer(-exp(lp1), H_grid))  # matrice n × T, scénario a0=1
S0_mat <- exp(outer(-exp(lp0), H_grid))  # matrice n × T, scénario a0=0

## 4. Courbes marginales : moyenne sur les individus (ligne = individu)
S1_marg <- colMeans(S1_mat)
S0_marg <- colMeans(S0_mat)

head(data.frame(t     = round(t_grid, 2),
                S_a1  = round(S1_marg, 3),
                S_a0  = round(S0_marg, 3)))
```

Étape 4 - Visualisation des courbes de survie contrefactuelles

Tracez les deux courbes de survie marginales contrefactuelles sur le même graphique.

```
#| context: interactive
## Tracez S1_marg en fonction de t_grid avec plot(..., type = "s") pour une courbe
## en escalier (adaptée aux estimateurs de survie discrets)
## Ajoutez S0_marg avec lines(), puis une légende
```

{Correction :}

```
## type = "s" : courbe en escalier, adaptée aux estimateurs de survie discrets
plot(t_grid, S1_marg, type = "s", col = "blue", lwd = 2,
     ylim = c(0, 1),
     xlab = "Temps (années)",
     ylab = "Probabilité de survie",
     main = "G-computation (Cox) - Courbes contrefactuelles")
lines(t_grid, S0_marg, type = "s", col = "red", lwd = 2)
legend("bottomleft",
       legend = expression(
         "Scénario " ~ a[0] * "=1 (tous exposés)",
         "Scénario " ~ a[0] * "=0 (aucun exposé)"
       ),
       col = c("blue", "red"), lty = 1, lwd = 2, bty = "n")
```

Étape 5 - Différence de survie à $t = 3$ ans

Calculez $\hat{S}^{a_0=1}(3)$, $\hat{S}^{a_0=0}(3)$, et la différence de survie à 3 ans. Comparez avec l'ATE obtenu par G-computation logistique.

```
#| context: interactive
## Survie marginale à t = 3 sous chaque scénario, et différence
```

{Correction :}

```
## t_grid ne contient pas nécessairement exactement t=3 : on prend
## le dernier temps d'événement inférieur ou égal à 3
idx3 <- max(which(t_grid <= 3))

S1_3 <- S1_marg[idx3]
S0_3 <- S0_marg[idx3]

cat("S^(a0=1)(3) =", round(S1_3, 3), "\n")
cat("S^(a0=0)(3) =", round(S0_3, 3), "\n")
cat("Différence de survie à 3 ans :", round(S1_3 - S0_3, 3), "\n")
cat("Rapport de survie à 3 ans    :", round(S1_3 / S0_3, 3), "\n")
```

Discussion - comparaison avec l'approche binaire

À titre indicatif, avec ce jeu de données : $\hat{S}^{a_0=1}(3) \approx 0.828$, $\hat{S}^{a_0=0}(3) \approx 0.641$, différence de survie ≈ 0.187.

Les deux G-computations (logistique et Cox) ciblent en principe le même estimand : $P(T^{a_0} \leq 3)$, ce qui correspond à $1 - S^{a_0}(3)$. En pratique, la différence de survie à $t = 3$ obtenue par les deux méthodes est souvent proche, mais peut légèrement différer car :

- **la version Cox utilise toute l'information temporelle (pas seulement le statut binaire à 3 ans) ;**
- **la version Cox traite correctement la censure : un individu censuré à $t = 1$ an ne contribue qu'à l'estimation jusqu'à son temps de censure, sans être compté comme "survivant" ou "décédé" à 3 ans ;**
- **les deux modèles imposent des hypothèses fonctionnelles différentes (effets proportionnels des hazards pour Cox, effets log-linéaires sur la cote pour la logistique).**

La version Cox est donc plus rigoureuse et constitue l'approche standard pour la G-computation avec un critère de survie.

Intervalle de confiance par bootstrap (modèle de Cox)

Estimez un intervalle de confiance à 95% par bootstrap ($B = 200$ ré-échantillons) pour la différence de survie à $t = 3$ ans.

```
#| context: interactive
set.seed(42)
B <- 200
S1_boot <- numeric(B)
S0_boot <- numeric(B)

for (b in 1:B) {
  ## 1. Ré-échantillonner df_base avec remise

  ## 2. Ajuster le modèle de Cox avec interaction sur l'échantillon bootstrap

  ## 3. Extraire basehaz et prédicteurs linéaires sous chaque scénario

  ## 4. Calculer S^(a0=1)(3) et S^(a0=0)(3) et les stocker
  S1_boot[b] <- ...
  S0_boot[b] <- ...
}

diff_boot <- S1_boot - S0_boot
cat("IC 95% :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

{Correction :}

```
set.seed(42)
B <- 200
S1_boot <- numeric(B)
S0_boot <- numeric(B)

for (b in 1:B) {
  ## 1. Ré-échantillonnage avec remise
```

```r
  idx <- sample(nrow(df_base), replace = TRUE)
  db  <- df_base[idx, ]

  ## 2. Modèle de Cox avec interaction sur l'échantillon bootstrap
  m <- suppressWarnings(coxph(Surv(time, D) ~ A0 * (X + L0), data = db))

  ## 3. Bases contrefactuelles, hazard de base et prédicteurs linéaires
  db1 <- db; db1$A0 <- 1
  db0 <- db; db0$A0 <- 0
  bhb  <- suppressWarnings(basehaz(m, centered = FALSE))
  lp1b <- predict(m, newdata = db1, type = "lp")
  lp0b <- predict(m, newdata = db0, type = "lp")

  ## 4. H0(3) : dernier hazard cumulatif <= 3 (0 si aucun événement avant 3 ans)
  H_3b <- if (any(bhb$time <= 3)) tail(bhb$hazard[bhb$time <= 3], 1) else 0

  ## S_i(3) = exp(-H0(3) * exp(lp_i)), puis moyenne sur les individus
  S1_boot[b] <- mean(exp(-H_3b * exp(lp1b)))
  S0_boot[b] <- mean(exp(-H_3b * exp(lp0b)))
}

diff_boot <- S1_boot - S0_boot
## L'estimée est celle calculée avant le bootstrap (S1_3 - S0_3)
## Le bootstrap ne sert qu'à l'intervalle de confiance
cat("Différence de survie à 3 ans (G-computation Cox) :\n")
cat("  Estimée :", round(S1_3 - S0_3, 3), "\n")
cat("  IC 95%  :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

---

2 IPTW

2.1 Objectif - Question 1 (suite)

Cette partie répond à la même Question 1 que la G-computation : estimer les courbes de survie contrefactuelles $S^{a_0=1}(t)$ et $S^{a_0=0}(t)$ ajustées sur les facteurs de confusion X et L_0.

L'IPTW adopte une stratégie opposée à la G-computation : au lieu de modéliser le résultat Y, il modélise l'exposition A_0 conditionnellement aux covariables, puis pondère les individus pour créer une pseudo-population équilibrée. Une fois les poids calculés, l'analyse de survie est directement réalisée par Kaplan-Meier pondéré, ce qui traite correctement la censure, de même que la G-computation de la courbe de survie présentée dans le bonus de la Partie 1. La simplification binaire de la Partie 1 avait été faite uniquement pour des raisons pédagogiques.

> **Note**
>
> **Lien avec la Partie 1 : la G-computation avait traité D comme une variable binaire (décédé avant 3 ans, oui/non) pour simplifier. Avec l'IPTW, l'implémentation est tout aussi simple avec le critère censuré. Des notes repliables proposent le code et les résultats équivalents en version binaire, pour comparer directement les deux méthodes à la fin.**

La variable `df` est disponible dans les chunks interactifs, avec `A0` et `L0` déjà propagées.

2.2 Étape 1 - Estimer le score de propension

Comme pour la G-computation, on travaille d'abord sur une base une ligne par individu (`df_base`) pour estimer le score de propension et les poids IPTW, puis on fusionne les poids dans `df` pour l'analyse de survie pondérée.

Le score de propension est $e_i = P(A_0 = 1 \mid X = x_i,\ L_0 = l_i)$, estimé par régression logistique sur les données de la première visite.

Créez `df_base` (une ligne par individu), estimez le modèle de propension `mod.ps` et stockez les probabilités prédites dans `df_base$ps`.

```
#| context: interactive
## Base une ligne par individu
df_base <- df |>
  group_by(id) |>
  summarise(A0 = first(A), L0 = first(L), X = first(X)) |>
  ungroup()

## Modèle logistique sur df_base

## Prédire ps dans df_base
```

{Correction :}

```r
## Base une ligne par individu
df_base <- df |>
  group_by(id) |>
  summarise(A0 = first(A), L0 = first(L), X = first(X)) |>
  ungroup()

## Modèle de propension : on modélise l'exposition A0 en fonction des covariables
## (à la différence de la G-computation qui modélisait l'outcome D)
mod.ps <- glm(A0 ~ X + L0, data = df_base, family = "binomial")

## type = "response" : probabilités prédites P(A0=1 | X, L0), pas le log-odds
df_base$ps <- predict(mod.ps, type = "response")

## Distribution du PS par groupe
summary(df_base$ps[df_base$A0 == 1])
summary(df_base$ps[df_base$A0 == 0])
```

Visualisez la distribution du score de propension selon A_0 pour vérifier le chevauchement (positivité) :

```r
#| context: interactive
## Tracez les distributions du PS dans les deux groupes sur le même graphique
## pour vérifier le chevauchement (positivité)
```

{Correction :}

```r
## Vérifier le chevauchement : si les deux distributions sont bien distinctes,
## la positivité est menacée et les poids seront instables
hist(df_base$ps[df_base$A0 == 1], breaks = 20, col = "#AC182E80",
     main = "Distribution du score de propension",
     xlab = "Score de propension", xlim = c(0, 1))
hist(df_base$ps[df_base$A0 == 0], breaks = 20, col = "#1D276980", add = TRUE)
legend("topright", legend = c("A0 = 1", "A0 = 0"),
       fill = c("#AC182E80", "#1D276980"), bty = "n")
```

2.3 Étape 2 - Calculer les poids IPTW

On calcule les poids non stabilisés $w_i = A_0/e_i + (1-A_0)/(1-e_i)$ et les poids stabilisés $w_i^s = P(A_0)/e_i$ si $A_0 = 1$, $P(A_0 = 0)/(1 - e_i)$ si $A_0 = 0$, sur `df_base`. On fusionne ensuite les poids dans `df` pour l'analyse de survie pondérée.

Calculez `df_base$iptw` (non stabilisé) et `df_base$iptw.s` (stabilisé), puis fusionnez dans `df`.

```
#| context: interactive
## Poids non stabilisés

## Poids stabilisés (numérateur = P(A0) marginal)

## Distribution des poids

## Fusion dans df
```

{Correction :}

```
## Poids non stabilisés : w = 1/ps si exposé, 1/(1-ps) si non exposé
df_base$iptw <- (df_base$A0 == 1) / df_base$ps +
                (df_base$A0 == 0) / (1 - df_base$ps)

## Numérateur des poids stabilisés : probabilité marginale d'exposition
## (sans covariables) - réduit la variance des poids par rapport aux non stabilisés
p.A1 <- mean(df_base$A0)
p.A0 <- 1 - p.A1

## Poids stabilisés : w^s = P(A0) / ps si exposé, P(1-A0) / (1-ps) si non exposé
df_base$iptw.s <- ifelse(df_base$A0 == 1,
                         p.A1 / df_base$ps,
                         p.A0 / (1 - df_base$ps))

## Les poids stabilisés ont une moyenne proche de 1 : vérifier des valeurs
## extrêmes (> 10-20) qui signaleraient un problème de positivité
cat("Poids non stabilisés - moyenne:", round(mean(df_base$iptw), 3),
    " / max:", round(max(df_base$iptw), 2), "\n")
cat("Poids stabilisés     - moyenne:", round(mean(df_base$iptw.s), 3),
    " / max:", round(max(df_base$iptw.s), 2), "\n")

## Fusionner dans df (format long) pour l'analyse de survie pondérée
df <- df |> left_join(df_base |> select(id, ps, iptw, iptw.s), by = "id")
```

Note

Règle pratique : les poids stabilisés ont une moyenne proche de 1 et une variance plus faible. Ils sont généralement préférés en pratique. Si certains poids sont très élevés (> 10–20), c'est un signe de violation de la positivité ou d'un modèle de propension mal spécifié.

2.4 Étape 3 - Vérifier l'équilibre

Avant toute analyse, il faut s'assurer que la pondération a rétabli l'équilibre entre les groupes $A_0 = 1$ et $A_0 = 0$ sur les covariables X et L_0.

Utilisez `cobalt::bal.tab()` et `cobalt::love.plot()` pour comparer les différences standardisées avant et après pondération.

```
#| context: interactive
library(cobalt)

## Utiliser bal.tab() pour calculer les différences standardisées avant/après pondération
## Visualiser avec love.plot() - le seuil conventionnel d'équilibre est 0,1
```

{Correction :}

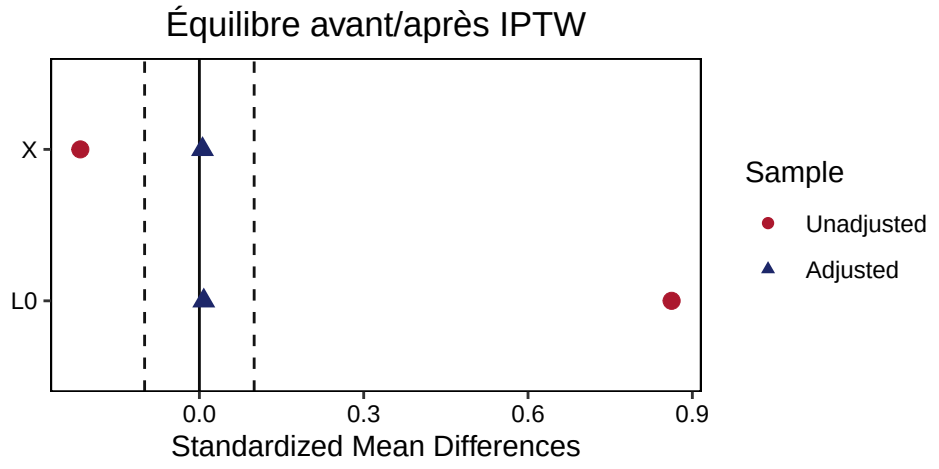
```
library(cobalt)

## bal.tab() : différences standardisées (SMD) pour chaque covariable
## binary = "std" : SMD pour les variables binaires, un = TRUE : affiche aussi avant pondération
bal <- bal.tab(A0 ~ X + L0,
               data    = df_base,
               weights = df_base$iptw.s,
               method  = "weighting",
               binary  = "std",
               un      = TRUE)
bal

## Love plot : SMD < 0,1 après pondération indique un équilibre satisfaisant
love.plot(bal,
          thresholds = c(m = 0.1),
          colors     = c("#AC182E", "#1D2769"),
          shapes     = c("circle", "triangle"),
          title      = "Équilibre avant/après IPTW")
```

Les différences standardisées après pondération sont-elles inférieures à 0,1 (seuil conventionnel) ?

Réponse



Si les SMD après pondération sont $< 0{,}1$ pour toutes les covariables, l'équilibre est satisfaisant. Un SMD résiduel $> 0{,}1$ suggère une mauvaise spécification du modèle de propension (variables manquantes, transformations nécessaires, interactions, etc.).
Rappel : n'utilisez pas de p-values pour évaluer l'équilibre : elles dépendent de la taille d'échantillon et ne mesurent pas la comparabilité des groupes.

2.5 Étape 4 - Analyse de survie pondérée (Kaplan-Meier)

Estimez les courbes de Kaplan-Meier pondérées par `iptw.s`, tracez-les et obtenez la différence de survie à 3 ans.

```
#| context: interactive
## Estimer les courbes de Kaplan-Meier pondérées par iptw.s avec survfit()
## Tracer les deux courbes et extraire la survie à 3 ans dans chaque groupe
```

{Correction :}

```
## Surv(T.start, T.stop, D) : format comptage (plusieurs lignes par individu)
## weights = iptw.s : chaque observation est pondérée par son poids IPTW stabilisé
km.iptw <- survfit(Surv(T.start, T.stop, D) ~ A0,
                   data    = df,
                   weights = iptw.s)

plot(km.iptw,
     col  = c("#1D2769", "#AC182E"), lwd = 2, conf.int = FALSE,
     xlab = "Temps (années)", ylab = "Probabilité de survie",
     main = "Kaplan-Meier pondéré (IPTW stabilisé)")
legend("bottomleft",
       legend = c("A0 = 0 (non-exposés)", "A0 = 1 (exposés)"),
       col = c("#1D2769", "#AC182E"), lwd = 2, bty = "n")

## Différence de survie à 3 ans (S^(a0=1)(3) - S^(a0=0)(3))
s3 <- summary(km.iptw, times = 3)$surv
cat("S(3 | a0=0)                        =", round(s3[1], 3), "\n")
cat("S(3 | a0=1)                        =", round(s3[2], 3), "\n")
cat("Diff. survie = S(a0=1) - S(a0=0)   =", round(s3[2] - s3[1], 3), "\n")
```

La différence de survie à 3 ans $S^{a_0=1}(3) - S^{a_0=0}(3)$ après IPTW est 0.167.

2.6 Comparaison G-computation vs IPTW

Les deux méthodes spécifient des modèles distincts pour répondre à la même question causale :

	G-computation (Partie 1)	IPTW (cette partie)
Modèle estimé	**Outcome :** $E(D \mid A_0, X, L_0)$	**Exposition :** $P(A_0 = 1 \mid X, L_0)$
Critère utilisé	**Binaire** ***(simplification)***	**Survie censurée** ***(complet)***
Mesure d'effet	$E(D^1) - E(D^0)$ **(diff. de risque de décès)**	$S^{a_0=1}(3) - S^{a_0=0}(3)$ **(diff. de survie)**
Estimée	**-0.135**	**0.167**

Puisque $P(D = 1) = 1 - S(3)$, la différence de risque de décès et la différence de survie sont de signes opposés et de valeur absolue proche. L'exercice ci-dessous permet de le vérifier en calculant l'IPTW sur critère binaire (comparable à la G-computation de la Partie 1).

À partir de `df_base` (une ligne par individu, avec `D` final et `iptw.s`), calculez l'ATE par IPTW avec critère binaire, et vérifiez sa cohérence avec la différence de survie du KM pondéré.

> **Avertissement**
>
> **`km.iptw` doit avoir été calculé à l'Étape 4 pour que ce chunk fonctionne.**

```
#| context: interactive
## df_base contient déjà une ligne par individu (avec D final et iptw.s)
## (ajoutez D final si nécessaire : df_base$D <- df |> group_by(id) |> summarise(D=last(D)) |> pull(D))

## Ajoutez D final à df_base, puis calculez m1_bin et m0_bin
## avec weighted.mean() pondéré par iptw.s
## Comparez l'ATE binaire avec la différence de survie du KM pondéré :
## les deux doivent être de signes opposés (risque de décès vs survie)
s3 <- summary(km.iptw, times = 3)$surv
```

{Correction :}

```
## D final : statut décès à la fin du suivi (dernière ligne par individu)
df_base$D <- df |> group_by(id) |> summarise(D = last(D)) |> pull(D)

## Moyenne pondérée de D dans chaque groupe : équivalent IPTW du critère binaire
m1_bin <- weighted.mean(df_base$D[df_base$A0 == 1],
                        df_base$iptw.s[df_base$A0 == 1])
m0_bin <- weighted.mean(df_base$D[df_base$A0 == 0],
                        df_base$iptw.s[df_base$A0 == 0])
ate_iptw_bin <- m1_bin - m0_bin

cat("=== IPTW (critère binaire) ===\n")
cat("E(D^1)                        =", round(m1_bin, 3), "\n")
cat("E(D^0)                        =", round(m0_bin, 3), "\n")
cat("ATE = E(D^1) - E(D^0)         =", round(ate_iptw_bin, 3), "\n\n")

s3 <- summary(km.iptw, times = 3)$surv
cat("=== IPTW (KM pondéré, critère censuré) ===\n")
cat("S(3 | a0=1)                   =", round(s3[2], 3), "\n")
cat("S(3 | a0=0)                   =", round(s3[1], 3), "\n")
cat("Diff. survie = S(1) - S(0)    =", round(s3[2] - s3[1], 3), "\n\n")
```

```r
cat("=== Vérification du lien ATE  -diff(survie) ===\n")
cat("ATE + diff(survie) =", round(ate_iptw_bin + (s3[2] - s3[1]), 3),
    "  (doit être proche de 0)\n")
```

À retenir

Les estimations obtenues sont :
- **G-computation (binaire, Partie 1) : ATE ≈ -0.135**
- **IPTW (binaire) : ATE ≈ -0.15**
- **IPTW (KM pondéré, survie) : diff. de survie ≈ 0.167**

Critère censuré vs binaire : la différence de survie par KM pondéré est l'estimateur de référence de cette partie, car il exploite toute l'information temporelle et traite correctement la censure. La version binaire est proposée uniquement pour le parallèle avec la Partie 1.

La Partie 3 passe à la Question 2 : l'effet de l'exposition *maintenue* tout au long du suivi.

2.7 BONUS - Intervalle de confiance par bootstrap

Note

Cette section est à faire chez vous, à votre rythme. Elle n'est pas traitée en TP.

Le bootstrap sert ici uniquement à estimer un intervalle de confiance autour de la différence de survie calculée à l'Étape 4 : il ne modifie pas l'estimation elle-même.

Pour les données en format long, le bootstrap ré-échantillonne des individus (pas des lignes) afin de préserver la structure temporelle. Chaque individu sélectionné reçoit un nouvel identifiant unique pour éviter les doublons.

Estimez un intervalle de confiance à 95% par bootstrap ($B = 200$ ré-échantillons) pour la différence de survie à 3 ans estimée par IPTW.

```r
#| context: interactive
set.seed(123)
B    <- 200
diff_boot <- numeric(B)
ids  <- unique(df$id)

for (b in 1:B) {
  ## 1. Ré-échantillonner les individus avec remise (cluster bootstrap)
  ##    Attribuer de nouveaux IDs uniques à chaque copie

  ## 2. Reconstruire df_base_b et estimer le score de propension

  ## 3. Calculer les poids IPTW stabilisés et les fusionner dans df_b

  ## 4. KM pondéré et stocker la différence de survie à 3 ans
```

```
}

## L'estimée est celle de l'Étape 4, pas la moyenne des bootstrap
cat("Différence de survie à 3 ans (IPTW) :\n")
cat("  Estimée :", round(s3[2] - s3[1], 3), "\n")
cat("  IC 95%  :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

{Correction :}

```
set.seed(123)
B    <- 200
diff_boot <- numeric(B)
ids  <- unique(df$id)

for (b in 1:B) {
  ## 1. Ré-échantillonnage par individu avec nouveaux IDs uniques
  ids_b <- sample(ids, replace = TRUE)
  df_b  <- do.call(rbind, lapply(seq_along(ids_b), function(j) {
    rows    <- df[df$id == ids_b[j], ]
    rows$id <- j   # nouvel ID unique pour éviter les doublons
    rows
  }))

  ## 2. Base une ligne par individu et score de propension
  df_base_b    <- df_b |>
    group_by(id) |>
    summarise(A0 = first(A), L0 = first(L), X = first(X)) |>
    ungroup()
  mod_b        <- glm(A0 ~ X + L0, data = df_base_b, family = "binomial")
  df_base_b$ps <- predict(mod_b, type = "response")

  ## 3. Poids IPTW stabilisés et fusion dans df_b
  ## (on supprime d'abord les anciens poids hérités de df pour éviter les conflits)
  p.A1_b           <- mean(df_base_b$A0)
  df_base_b$iptw.s <- ifelse(df_base_b$A0 == 1,
                             p.A1_b / df_base_b$ps,
                             (1 - p.A1_b) / (1 - df_base_b$ps))
  df_b <- df_b |>
    select(-any_of(c("ps", "iptw", "iptw.s"))) |>
    left_join(df_base_b |> select(id, iptw.s), by = "id")

  ## 4. KM pondéré et différence de survie à 3 ans
  km_b         <- survfit(Surv(T.start, T.stop, D) ~ A0, data = df_b, weights = iptw.s)
  s3_b         <- summary(km_b, times = 3)$surv
  diff_boot[b] <- s3_b[2] - s3_b[1]
}

## L'estimée est celle de l'Étape 4, pas la moyenne des bootstrap
cat("Différence de survie à 3 ans (IPTW) :\n")
cat("  Estimée :", round(s3[2] - s3[1], 3), "\n")
cat("  IC 95%  :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

3 IPCW

3.1 Objectif - Question 2

Les Parties 1 et 2 ont estimé l'effet de l'*initiation* de l'exposition A_0, sans se préoccuper de ce qui se passe ensuite (les patients peuvent arrêter ou débuter le traitement par la suite). C'est l'analogue-ITT.

Cette partie répond à une nouvelle question : quel serait l'effet si les patients avaient *maintenu* leur stratégie d'exposition tout au long du suivi ?

$$S^{\bar{a}=1}(t) = P(T^{\bar{a}=1} > t) \quad \textbf{et} \quad S^{\bar{a}=0}(t) = P(T^{\bar{a}=0} > t)$$

Stratégie : grouper les individus selon A_0, puis censurer artificiellement tout individu qui dévie de sa stratégie initiale. Cette censure est très probablement informative (les patients qui changent de traitement diffèrent des autres) → on corrige par IPCW. Les poids IPTW de la Partie 2 sont conservés pour l'équilibre initial.

3.2 Note sur la session R

Les variables suivantes sont disponibles dans vos chunks interactifs (reconstruites dans le contexte de setup) :

- **`df` : base avec `A0`, `L0` propagés**
- **`df$ps` : score de propension $P(A_0 = 1 \mid X, L_0)$**
- **`df$iptw.s` : poids IPTW stabilisés (de la Partie 2)**

3.3 Étape 1 - Définir les deux groupes de stratégie

Créez `df.1` (individus avec `A0 == 1`) et `df.0` (individus avec `A0 == 0`).

```
#| context: interactive
## Groupe stratégie "toujours exposé"

## Groupe stratégie "jamais exposé"
```

{Correction :}

```
## On sépare les individus selon leur stratégie initiale A0 :
## les poids IPCW et la censure artificielle seront calculés séparément
## dans chaque groupe (déviation = arrêt pour df.1, initiation pour df.0)
df.1 <- df[df$A0 == 1, ]
df.0 <- df[df$A0 == 0, ]

cat("Individus avec A0=1 :", length(unique(df.1$id)), "\n")
cat("Individus avec A0=0 :", length(unique(df.0$id)), "\n")
```

3.4 Étape 2 - Censure artificielle dans le groupe A0 = 1

Pour la stratégie $\bar{a} = 1$, un individu dévie dès qu'il arrête le traitement ($A = 0$ à une visite ultérieure).

Dans `df.1`, créez `switchA` : 0 tant que l'individu suit la stratégie, 1 à la première déviation. Conservez toutes les lignes jusqu'à `switchA  1` (on garde la ligne de déviation pour estimer le modèle de poids).

```
#| context: interactive
## Somme cumulée de A=1 au fil du temps

## switchA = 0 si A=1 à toutes les visites jusqu'ici, 1 sinon (puis cumprod)

## Garder switchA <= 1
```

{Correction :}

```
df.1 <- df.1 |>
  group_by(id) |>
  mutate(
    ## cumsumA compte le nombre de visites où A=1 depuis le début
    cumsumA = cumsum(A == 1),
    ## Si l'individu n'a jamais dévié, cumsumA = nombre de visites écoulées = T.start + 1
    ## Si cumsumA < T.start + 1, il y a eu au moins une visite avec A=0 -> déviation
    switchA = if_else(cumsumA == T.start + 1, 0L, 1L),
    ## cumsum(switchA) : une fois la déviation détectée, switchA reste >= 1
    switchA = cumsum(switchA)
  ) |>
  filter(switchA <= 1) |>  # garder jusqu'à la première déviation incluse
  ungroup()

table(df.1$switchA)
```

3.5 Étape 3 - Modèle de déviation et poids IPCW (groupe A0 = 1)

On estime la probabilité de ne pas dévier à chaque visite par régression logistique poolée (*pooled logistic regression*), puis on calcule les poids IPCW :

$$W_i(t) = \prod_{k=0}^{t} \frac{1}{P(C_{ik} = 0 \mid X_i, L_{ik})} \qquad t = 0, 1, 2$$

Le signe $\prod$ indique un produit : le poids s'accumule à chaque visite. Par exemple, à $t = 2$:

$$W_i(2) = \frac{1}{P(C_{i0} = 0 \mid X_i, L_{i0})} \times \frac{1}{P(C_{i1} = 0 \mid X_i, L_{i1})} \times \frac{1}{P(C_{i2} = 0 \mid X_i, L_{i2})}$$

Pour les poids stabilisés, le numérateur est la probabilité marginale de ne pas dévier (estimée sans covariables) :

$$W_i^s(t) = \prod_{k=0}^{t} \frac{P(C_{ik} = 0)}{P(C_{ik} = 0 \mid X_i, L_{ik})}$$

Ajustez `wt.mod.1 <- glm(switchA ~ as.factor(T.start) + X + L, ...)` sur `df.1`, stockez `wt.denom` = $P(\textit{switch} = 0)$ prédit. Pour les poids stabilisés, estimez un second modèle sans covariables (`wt.mod.1.num`). Supprimez les lignes `switchA == 1`. Calculez `wt` (non stabilisé) et `wt.s` (stabilisé).

```
#| context: interactive
## Modèle de déviation (régression logistique poolée)

## Probabilité de ne PAS dévier (dénominateur)

## Modèle sans covariables (numérateur des poids stabilisés)

## Supprimer les lignes de déviation (switchA == 1)

## Poids IPCW non stabilisés et stabilisés
```

{Correction :}

```
## Régression logistique poolée : modélise P(switch=1) à chaque visite
## as.factor(T.start) : effet du temps, X et L : covariables de confusion
wt.mod.1 <- glm(switchA ~ as.factor(T.start) + X + L,
                family = "binomial", data = df.1)
## 1 - predict(...) : probabilité de NE PAS dévier à cette visite
df.1$wt.denom <- 1 - predict(wt.mod.1, type = "response", newdata = df.1)

## Numérateur : même modèle sans covariables (probabilité marginale de ne pas dévier)
wt.mod.1.num  <- glm(switchA ~ as.factor(T.start),
                     family = "binomial", data = df.1)
df.1$wt.num   <- 1 - predict(wt.mod.1.num, type = "response", newdata = df.1)

## Supprimer la ligne de déviation : on ne garde que les périodes sans déviation
```

```r
df.1 <- df.1[df.1$switchA == 0, ]

## cumprod : produit cumulé des termes période par période -> poids croissant dans le temps
df.1 <- df.1 |>
  group_by(id) |>
  mutate(wt   = cumprod(1 / wt.denom),          # non stabilisé
         wt.s = cumprod(wt.num / wt.denom)) |>  # stabilisé
  ungroup()

cat("Poids non stabilisés - moy:", round(mean(df.1$wt), 3),
    " / max:", round(max(df.1$wt), 2), "\n")
cat("Poids stabilisés     - moy:", round(mean(df.1$wt.s), 3),
    " / max:", round(max(df.1$wt.s), 2), "\n")
```

```
Warning: glm.fit : l'algorithme n'a pas convergé
Warning: glm.fit : l'algorithme n'a pas convergé
```

3.6 Étape 4 - Répéter pour le groupe A0 = 0

Reproduire les étapes 2 et 3 pour `df.0` (déviation = passer de $A = 0$ à $A = 1$). Empiler ensuite `df.1` et `df.0` dans `dfpp`.

```r
#| context: interactive
## Censure artificielle dans df.0 (déviation = A passe à 1)

## Modèle de déviation, wt.denom, suppression switchA==1, poids wt

## Empilement
## dfpp <- rbind(df.1, df.0)
```

{Correction :}

```r
## Même logique que pour df.1, mais la déviation est ici A=1 (passage de 0 à 1)
## cumsumA compte donc le nombre de visites où A=0
df.0 <- df.0 |>
  group_by(id) |>
  mutate(
    cumsumA = cumsum(A == 0),
    switchA = if_else(cumsumA == T.start + 1, 0L, 1L),
    switchA = cumsum(switchA)
  ) |>
  filter(switchA <= 1) |>
  ungroup()

## Modèle poolé de déviation dans df.0 (avec covariables)
wt.mod.0      <- glm(switchA ~ as.factor(T.start) + X + L,
                     family = "binomial", data = df.0)
df.0$wt.denom <- 1 - predict(wt.mod.0, type = "response", newdata = df.0)
```

```r
## Numérateur (sans covariables)
wt.mod.0.num  <- glm(switchA ~ as.factor(T.start),
                     family = "binomial", data = df.0)
df.0$wt.num   <- 1 - predict(wt.mod.0.num, type = "response", newdata = df.0)

df.0          <- df.0[df.0$switchA == 0, ]
df.0          <- df.0 |>
  group_by(id) |>
  mutate(wt   = cumprod(1 / wt.denom),
         wt.s = cumprod(wt.num / wt.denom)) |>
  ungroup()

## Empilement
dfpp <- rbind(df.1, df.0)
cat("Lignes dans dfpp :", nrow(dfpp), "\n")
```

```
Warning: glm.fit : l'algorithme n'a pas convergé
Warning: glm.fit : l'algorithme n'a pas convergé
```

3.7 Étape 5 - Poids combinés IPTW × IPCW

Les poids IPCW (`wt`) corrigent la censure artificielle informative. Mais ils ne rééquilibrent pas encore les caractéristiques initiales entre $A_0 = 1$ et $A_0 = 0$. Il faut combiner avec les poids IPTW estimés en Partie 2.

Calculez `dfpp$comb.wt = dfpp$iptw.s * dfpp$wt.s` (poids combinés stabilisés), puis vérifiez la distribution.

```r
#| context: interactive
## Poids combinés (IPTW stabilisé × IPCW stabilisé)

## Distribution
```

{Correction :}

```r
## IPTW corrige la confusion initiale, IPCW corrige la censure informative
## Les deux biais étant indépendants, leurs corrections se multiplient
dfpp$comb.wt <- dfpp$iptw.s * dfpp$wt.s

cat("Poids IPCW stabilisés - moy:", round(mean(dfpp$wt.s), 3),
    " / max:", round(max(dfpp$wt.s), 2), "\n")
cat("Poids combinés        - moy:", round(mean(dfpp$comb.wt), 3),
    " / max:", round(max(dfpp$comb.wt), 2), "\n")
```

Note

Pourquoi multiplier ? Les poids IPTW rééquilibrent les groupes sur les caractéristiques initiales (X, L_0). Les poids IPCW compensent la censure artificielle informative au cours du suivi (L_t, A_t). Les deux sources de biais sont indépendantes, donc leurs corrections se multiplient.

3.8 Étape 6 - Analyse de survie per-protocol

Tracez le Kaplan-Meier pondéré par `comb.wt` dans `dfpp`. Comparez avec l'analogue-ITT de la Partie 2.

```
#| context: interactive
## Estimer le KM pondéré par comb.wt dans dfpp, tracer les deux courbes
## et extraire la différence de survie à 3 ans (stratégie ā=1 moins ā=0)
```

{Correction :}

```
## dfpp contient les deux groupes après censure artificielle
## comb.wt = IPTW × IPCW : corrige à la fois la confusion et la censure informative
km.pp <- survfit(Surv(T.start, T.stop, D) ~ A0,
                 data    = dfpp,
                 weights = comb.wt)

plot(km.pp,
     col  = c("#1D2769", "#AC182E"), lwd = 2, conf.int = FALSE,
     xlab = "Temps (années)", ylab = "Probabilité de survie",
     main = "Kaplan-Meier per-protocol (IPTW × IPCW)")
legend("bottomleft",
       legend = expression(
         "Stratégie " ~ bar(a) * "=0 (jamais exposé)",
         "Stratégie " ~ bar(a) * "=1 (toujours exposé)"
       ),
       col = c("#1D2769", "#AC182E"), lwd = 2, bty = "n")

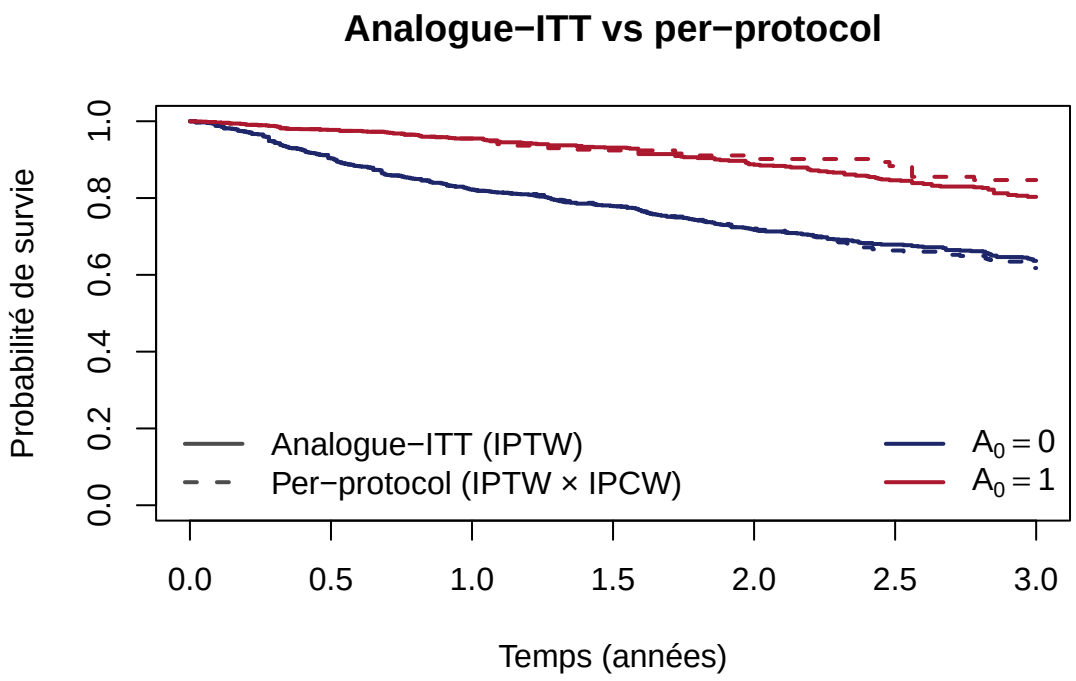
## Différence de survie à 3 ans (ā=1 moins ā=0)
s3.pp <- summary(km.pp, times = 3)$surv
cat("Survie à 3 ans - ā=0 :", round(s3.pp[1], 3), "\n")
cat("Survie à 3 ans - ā=1 :", round(s3.pp[2], 3), "\n")
cat("Différence S(ā=1) - S(ā=0) :", round(s3.pp[2] - s3.pp[1], 3), "\n")
```

La différence de survie à 3 ans $S^{\bar{a}=1}(3) - S^{\bar{a}=0}(3)$ (per-protocol) est 0.229.

3.9 Interprétation

Comparez l'effet per-protocol à l'effet analogue-ITT. Pourquoi peut-il différer ?

Réponse



— **L'analogue-ITT estime l'effet d'initier l'exposition A_0, quelle que soit la compliance ultérieure. Il dilue l'effet si certains exposés arrêtent leur traitement.**
— **L'analogue per-protocol estime l'effet de maintenir l'exposition tout au long du suivi. Il peut être plus fort (si l'exposition continue est nécessaire) ou différent pour d'autres raisons.**

Si les deux estimations convergent, l'arrêt du traitement n'a pas d'impact majeur sur la survie. Si elles divergent, la compliance joue un rôle.

Hypothèses supplémentaires nécessaires pour l'analogue-PP :

— **Échangeabilité conditionnelle au cours du temps : L_t capture tous les facteurs qui prédisent à la fois la déviation et le décès.**
— **Positivité au cours du temps : à chaque visite, chaque individu doit avoir une probabilité positive de continuer ou d'arrêter.**

3.10 BONUS - Intervalle de confiance par bootstrap

Note

Cette section est à faire chez vous, à votre rythme. Elle n'est pas traitée en TP. La boucle bootstrap est plus longue qu'en Parties 1 et 2 car elle refait toute la chaîne d'analyse (IPTW + censure artificielle + IPCW) à chaque itération.

Le bootstrap sert ici uniquement à estimer un intervalle de confiance autour de la différence de survie per-protocol calculée à l'Étape 6 : il ne modifie pas l'estimation elle-même.

Comme en Partie 2, le bootstrap ré-échantillonne des individus (pas des lignes) avec attribution de nouveaux identifiants uniques.

Estimez un intervalle de confiance à 95% par bootstrap ($B = 200$ ré-échantillons) pour la différence de survie à 3 ans per-protocol.

```
#| context: interactive
set.seed(42)
B    <- 200
diff_boot <- numeric(B)
ids  <- unique(df$id)

for (b in 1:B) {
  ## 1. Ré-échantillonner les individus avec remise, nouveaux IDs uniques

  ## 2. Recalculer IPTW (df_base_b -> mod_b -> iptw.s -> fusion dans df_b)

  ## 3. Censure artificielle et poids IPCW stabilisés pour df1_b (stratégie ā=1)

  ## 4. Censure artificielle et poids IPCW stabilisés pour df0_b (stratégie ā=0)

  ## 5. Empiler, poids combinés, KM per-protocol, stocker la différence à 3 ans
}

## L'estimée est celle de l'Étape 6, pas la moyenne des bootstrap
cat("Différence de survie à 3 ans (per-protocol) :\n")
cat("  Estimée :", round(s3.pp[2] - s3.pp[1], 3), "\n")
cat("  IC 95%  :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

{Correction :}

```
set.seed(42)
B    <- 200
diff_boot <- numeric(B)
ids  <- unique(df$id)

for (b in 1:B) {
  ## 1. Ré-échantillonnage par individu avec nouveaux IDs uniques
  ids_b <- sample(ids, replace = TRUE)
  df_b  <- do.call(rbind, lapply(seq_along(ids_b), function(j) {
    rows    <- df[df$id == ids_b[j], ]
    rows$id <- j
    rows
  }))

  ## 2. IPTW : score de propension et poids stabilisés
  df_base_b        <- df_b |>
    group_by(id) |>
    summarise(A0 = first(A), L0 = first(L), X = first(X)) |>
    ungroup()
  mod_b            <- glm(A0 ~ X + L0, data = df_base_b, family = "binomial")
  df_base_b$ps     <- predict(mod_b, type = "response")
  p.A1_b           <- mean(df_base_b$A0)
  df_base_b$iptw.s <- ifelse(df_base_b$A0 == 1,
                             p.A1_b / df_base_b$ps,
```

```
                              (1 - p.A1_b) / (1 - df_base_b$ps))
  ## Supprimer les anciens poids hérités de df avant de joindre les nouveaux
  df_b <- df_b |>
    select(-any_of(c("ps", "iptw", "iptw.s"))) |>
    left_join(df_base_b |> select(id, iptw.s), by = "id")

  ## 3. Censure artificielle et poids IPCW stabilisés pour la stratégie ā=1
  df1_b <- df_b[df_b$A0 == 1, ] |>
    group_by(id) |>
    mutate(cumsumA = cumsum(A == 1),
           switchA = if_else(cumsumA == T.start + 1, 0L, 1L),
           switchA = cumsum(switchA)) |>
    filter(switchA <= 1) |> ungroup()
  wt1_b          <- glm(switchA ~ as.factor(T.start) + X + L,
                        family = "binomial", data = df1_b)
  wt1_num_b      <- glm(switchA ~ as.factor(T.start),
                        family = "binomial", data = df1_b)
  df1_b$wt.denom <- 1 - predict(wt1_b, type = "response", newdata = df1_b)
  df1_b$wt.num   <- 1 - predict(wt1_num_b, type = "response", newdata = df1_b)
  df1_b <- df1_b[df1_b$switchA == 0, ] |>
    group_by(id) |>
    mutate(wt.s = cumprod(wt.num / wt.denom)) |> ungroup()

  ## 4. Censure artificielle et poids IPCW stabilisés pour la stratégie ā=0
  df0_b <- df_b[df_b$A0 == 0, ] |>
    group_by(id) |>
    mutate(cumsumA = cumsum(A == 0),
           switchA = if_else(cumsumA == T.start + 1, 0L, 1L),
           switchA = cumsum(switchA)) |>
    filter(switchA <= 1) |> ungroup()
  wt0_b          <- glm(switchA ~ as.factor(T.start) + X + L,
                        family = "binomial", data = df0_b)
  wt0_num_b      <- glm(switchA ~ as.factor(T.start),
                        family = "binomial", data = df0_b)
  df0_b$wt.denom <- 1 - predict(wt0_b, type = "response", newdata = df0_b)
  df0_b$wt.num   <- 1 - predict(wt0_num_b, type = "response", newdata = df0_b)
  df0_b <- df0_b[df0_b$switchA == 0, ] |>
    group_by(id) |>
    mutate(wt.s = cumprod(wt.num / wt.denom)) |> ungroup()

  ## 5. Poids combinés et KM per-protocol
  dfpp_b         <- rbind(df1_b, df0_b)
  dfpp_b$comb.wt <- dfpp_b$iptw.s * dfpp_b$wt.s
  km_b           <- survfit(Surv(T.start, T.stop, D) ~ A0,
                            data = dfpp_b, weights = comb.wt)
  s3_b           <- summary(km_b, times = 3)$surv
  diff_boot[b]   <- s3_b[2] - s3_b[1]
}

## L'estimée est celle de l'Étape 6, pas la moyenne des bootstrap
cat("Différence de survie à 3 ans (per-protocol) :\n")
cat("  Estimée :", round(s3.pp[2] - s3.pp[1], 3), "\n")
cat("  IC 95%  :", round(quantile(diff_boot, c(0.025, 0.975)), 3), "\n")
```

3.11 BONUS 2 - Clonage, censure, pondération : IPCW seul

> **Note**
>
> **Cette section illustre ce qui a été dit en cours : dans l'approche par clonage, l'IPCW seul suffit — sans IPTW. La clé est que le modèle IPCW inclut les observations au temps 0 où la déviation immédiate est exactement le complément du score de propension. Pour cela, les modèles doivent être ajustés séparément dans chaque groupe de clones : à T.start = 0, les effets de X et L sur la déviation sont de sens opposés entre les deux stratégies, et un modèle commun annulerait ces effets.**

```
##   Étape 1 : Cloner
## Chaque individu reçoit deux clones : un assigné à la stratégie ā=1, l'autre ā=0
df.c1 <- df; df.c1$strategy <- 1
df.c0 <- df; df.c0$strategy <- 0
df.clone <- rbind(df.c1, df.c0)

##   Étape 2 : Censure artificielle selon la stratégie assignée
df.clone <- df.clone |>
  group_by(id, strategy) |>
  mutate(
    cumsumA = if_else(strategy == 1, cumsum(A == 1), cumsum(A == 0)),
    switchA  = if_else(cumsumA == T.start + 1, 0L, 1L),
    switchA  = cumsum(switchA)
  ) |>
  filter(switchA <= 1) |>
  ungroup()

##   Étape 3 : IPCW seul, modèles séparés par groupe de clones
## À T.start = 0, strategy = 1 : déviation = I(A = 0) → P(pas de déviation | X, L) = P(A=1|X,L)
## À T.start = 0, strategy = 0 : déviation = I(A = 1) → P(pas de déviation | X, L) = P(A=0|X,L)
## Le terme T.start = 0 du modèle par groupe absorbe exactement le poids IPTW.
## Les modèles DOIVENT être séparés car les effets de X et L sont opposés.
dc1 <- df.clone[df.clone$strategy == 1, ]
dc0 <- df.clone[df.clone$strategy == 0, ]

wd1 <- glm(switchA ~ as.factor(T.start) + X + L, family = "binomial", data = dc1)
wn1 <- glm(switchA ~ as.factor(T.start),          family = "binomial", data = dc1)
dc1$wt.denom <- 1 - predict(wd1, type = "response", newdata = dc1)
dc1$wt.num   <- 1 - predict(wn1, type = "response", newdata = dc1)

wd0 <- glm(switchA ~ as.factor(T.start) + X + L, family = "binomial", data = dc0)
wn0 <- glm(switchA ~ as.factor(T.start),          family = "binomial", data = dc0)
dc0$wt.denom <- 1 - predict(wd0, type = "response", newdata = dc0)
dc0$wt.num   <- 1 - predict(wn0, type = "response", newdata = dc0)

df.clone <- rbind(dc1, dc0)
```

```r
df.clone <- df.clone[df.clone$switchA == 0, ] |>
  group_by(id, strategy) |>
  mutate(ipcw.s = cumprod(wt.num / wt.denom)) |>
  ungroup()

## KM pondéré IPCW seul (approche clonage)
km.clone <- survfit(Surv(T.start, T.stop, D) ~ strategy,
                    data = df.clone, weights = ipcw.s)
```

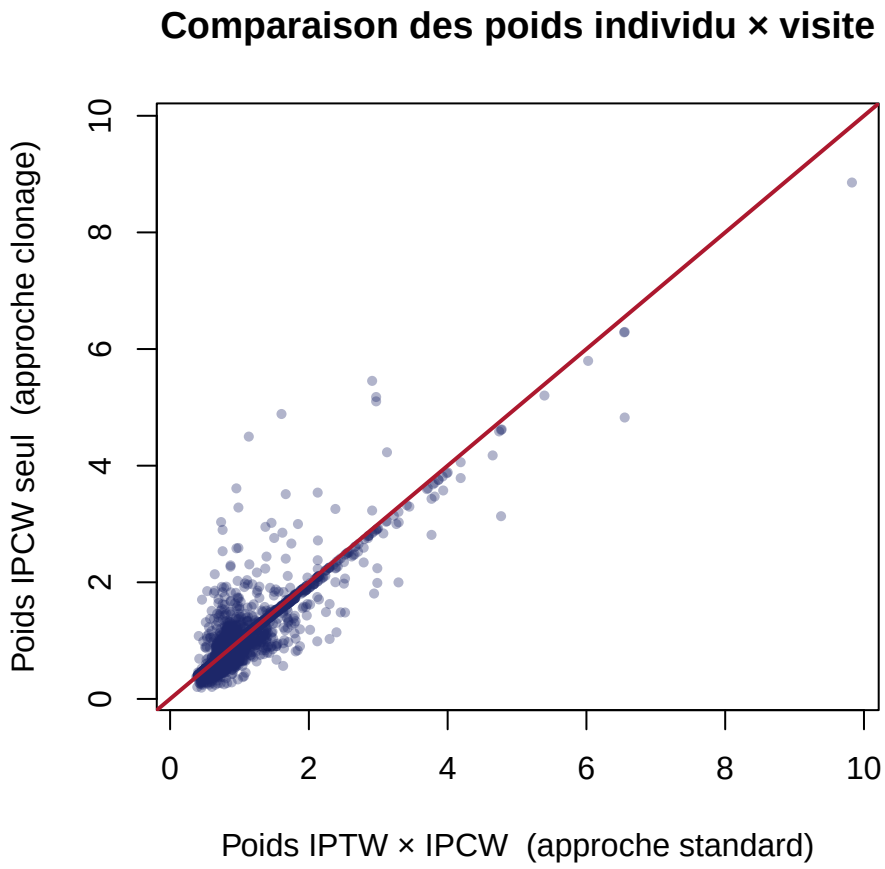
Comparaison des poids individu × visite

Les poids IPTW × IPCW (approche standard, `dfpp$comb.wt`) et les poids IPCW seul (approche clonage, `df.clone$ipcw.s`) doivent être alignés sur la diagonale si les deux approches estiment bien la même chose.

```r
comp <- merge(
  dfpp[, c("id", "T.start", "A0", "comb.wt")],
  df.clone[, c("id", "T.start", "strategy", "ipcw.s")],
  by.x = c("id", "T.start", "A0"),
  by.y = c("id", "T.start", "strategy")
)

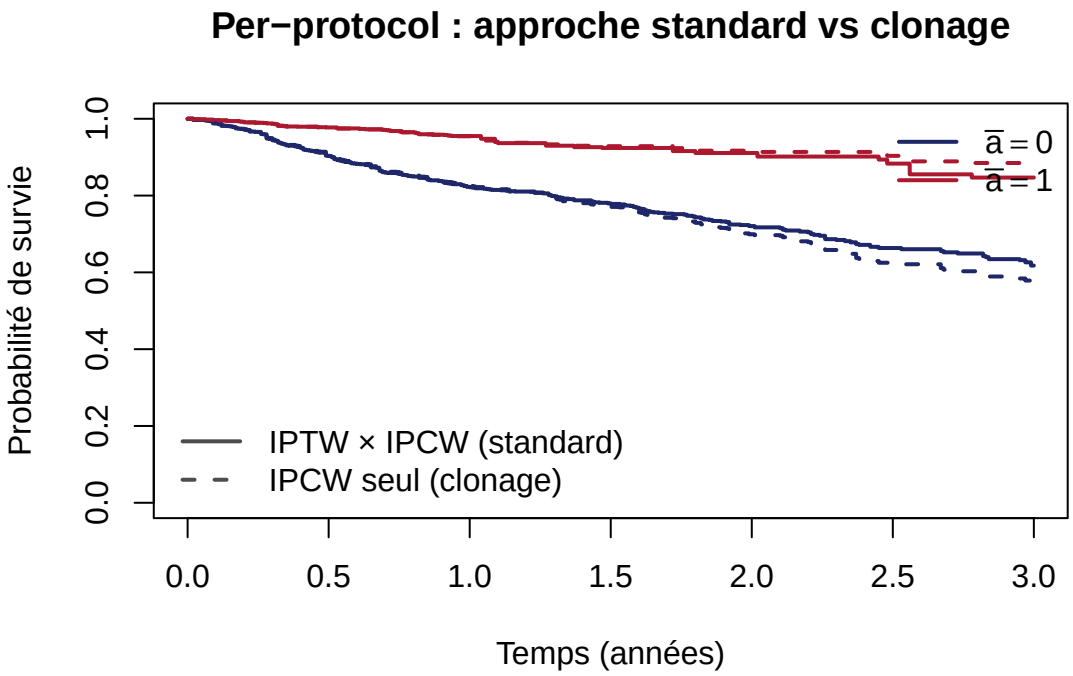
col0 <- "#1D2769"; col1 <- "#AC182E"
lim  <- range(c(comp$comb.wt, comp$ipcw.s))

plot(comp$comb.wt, comp$ipcw.s,
     xlim = lim, ylim = lim,
     xlab = "Poids IPTW × IPCW  (approche standard)",
     ylab = "Poids IPCW seul  (approche clonage)",
     pch = 16, col = adjustcolor(col0, alpha.f = 0.35), cex = 0.7,
     main = "Comparaison des poids individu × visite")
abline(0, 1, col = col1, lwd = 2)
```



Comparaison des courbes de survie

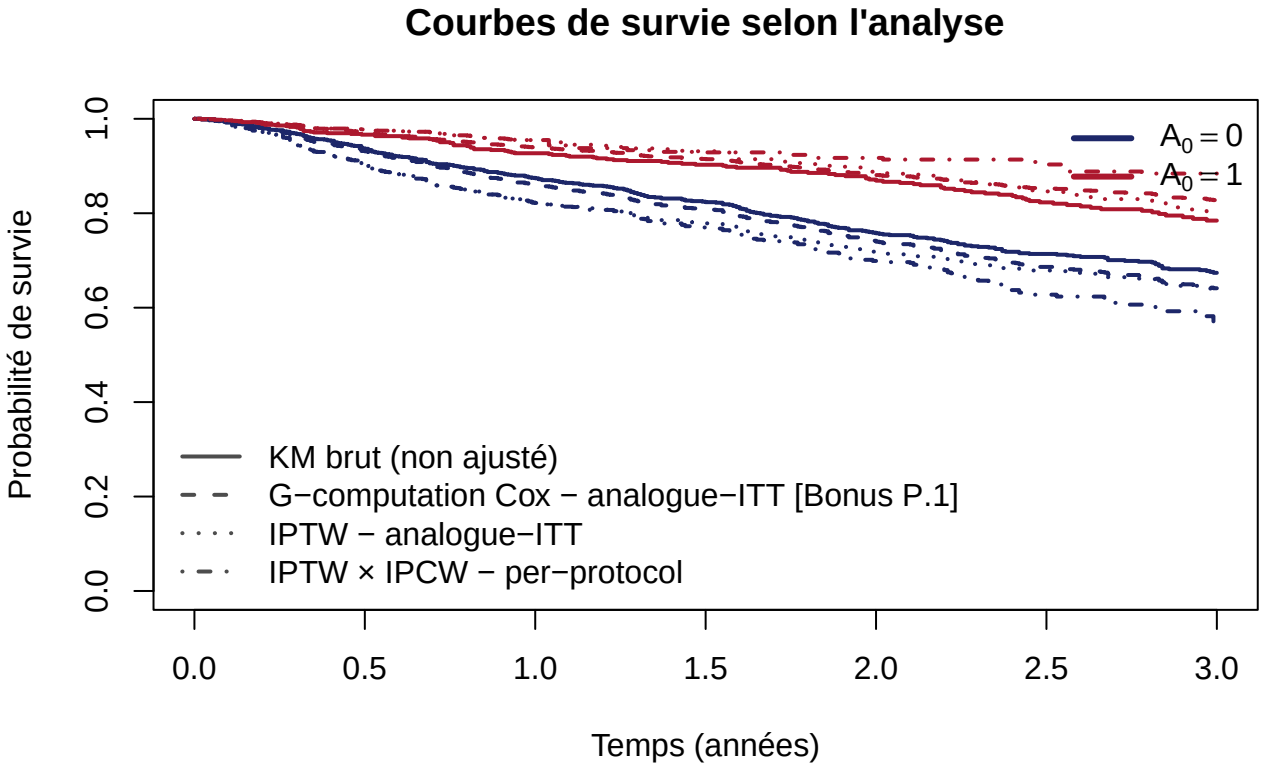
```
plot(km.pp,
     col = c(col0, col1), lwd = 2, lty = 1, conf.int = FALSE,
     ylim = c(0, 1), xlim = c(0, 3),
     xlab = "Temps (années)", ylab = "Probabilité de survie",
     main = "Per-protocol : approche standard vs clonage")
lines(km.clone, col = c(col0, col1), lwd = 2, lty = 2, conf.int = FALSE)
legend("bottomleft", lty = 1:2, lwd = 2, col = "gray30", bty = "n",
       legend = c("IPTW × IPCW (standard)", "IPCW seul (clonage)"))
legend("topright", lty = 1, lwd = 2, col = c(col0, col1), bty = "n",
       legend = c(expression(bar(a) == 0), expression(bar(a) == 1)))
```



Les courbes se superposent et les poids sont alignés sur la diagonale : les deux approches estiment bien le même estimand per-protocol. Le clonage rend l'IPTW redondant car il garantit l'équilibre initial entre les groupes par construction.

4 Conclusion

4.1 Comparaison graphique



4.2 Récapitulatif des estimations

Analyse	Estimand	Ajustement	S(3) - A =0	S(3) - A =1	Δ sur-vie
KM brut (non ajusté)	-	Aucun	0.674	0.784	0.111
Q1 - G-computation Cox [Bonus P.1]	Analogue-ITT : effet d'initier l'exposition	Confusion initiale (X, L) - modèle du résultat	0.641	0.828	0.187
Q1 - IPTW	Analogue-ITT : effet d'initier l'exposition	Confusion initiale (X, L) - modèle de l'exposition	0.636	0.803	0.167

Analyse	Estimand	Ajustement	S(3) - A =0	S(3) - A =1	Δ sur-vie
Q2 - IPTW × IPCW (per-protocol)	**Analogue per-protocol : effet de maintenir l'exposition**	**Confusion initiale + déviation de stratégie**	**0.568**	**0.884**	**0.316**

4.3 Discussion

Ce que nous apprenons de chaque analyse

- **Analyse brute : la différence de survie observée entre $A_0 = 1$ et $A_0 = 0$ n'est pas interprétable causalement. Les groupes ne sont pas comparables à l'inclusion (X, L_0 diffèrent).**
- **Analogue-ITT (G-computation Cox et IPTW - Q1) : après ajustement sur les caractéristiques initiales, ces deux méthodes estiment le même estimand : l'effet d'*initier* l'exposition au temps 0, quelle que soit la compliance ultérieure. C'est l'estimand le plus proche d'un essai clinique en intention de traiter. La G-computation ajuste via un modèle du résultat, l'IPTW via un modèle de l'exposition. Si les deux convergent, la confiance dans le résultat est renforcée.**
- **Analogue per-protocol (IPTW × IPCW - Q2) : en censurant artificiellement les individus qui dévient de leur stratégie et en corrigeant ce biais par IPCW, on estime l'effet d'*initier et de maintenir* l'exposition. Cet estimand est plus proche d'un essai per-protocol.**